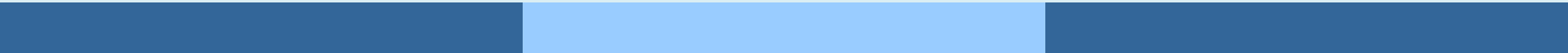


Chapter 8

DEADLOCKS



Chapter 7: Deadlocks

- System Model
- Deadlock Characterization
- Methods for Handling Deadlocks
- Deadlock Prevention
- Deadlock Avoidance
- Deadlock Detection
- Recovery from Deadlock

Chapter Objectives

- To develop a description of deadlocks, which prevent sets of concurrent processes from completing their tasks.
- To present a number of different methods for preventing or avoiding deadlocks in a computer system.

System Model

- System consists of resources.
- Resource types $R_1, R_2, \dots, R_m$
CPU cycles, memory space, I/O devices.
- Each resource type R_i has W_i instances.
- Each process utilizes a resource as follows:
 - **request**
 - **use**
 - **release**

Deadlock

- A deadlock is a situation in which two computer programs sharing the same resource are effectively preventing each other from accessing the resource, resulting in both programs ceasing to function.
- Here is the simplest example:

```
Program 1 requests resource A and receives it.  
Program 2 requests resource B and receives it.  
Program 1 requests resource B and is queued up, pending the release of B.  
Program 2 requests resource A and is queued up, pending the release of A.
```

Deadlock Characterization

Deadlock can arise if four conditions hold simultaneously.

- **Mutual exclusion:** only one process at a time can use a resource.
- **Hold and wait:** a process holding at least one resource is waiting to acquire additional resources held by other processes.
- **No preemption:** a resource can be released only voluntarily by the process holding it, after that process has completed its task.
- **Circular wait:** there exists a set $\{P_0, P_1, \dots, P_n\}$ of waiting processes such that P_0 is waiting for a resource that is held by P_1 , P_1 is waiting for a resource that is held by P_2 , ..., P_{n-1} is waiting for a resource that is held by P_n , and P_n is waiting for a resource that is held by P_0 .

Resource-Allocation Graph

A set of vertices V and a set of edges E .

- V is partitioned into two types:
 - $P = \{P_1, P_2, \dots, P_n\}$, the set consisting of all the processes in the system.
 - $R = \{R_1, R_2, \dots, R_m\}$, the set consisting of all resource types in the system.
- **request edge** – directed edge $P_i \rightarrow R_j$
- **assignment edge** – directed edge $R_j \rightarrow P_i$

Resource-Allocation Graph (Cont.)

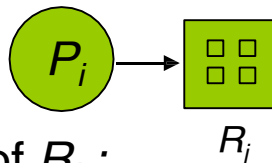
- Process:



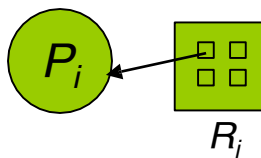
- Resource Type with 4 instances:



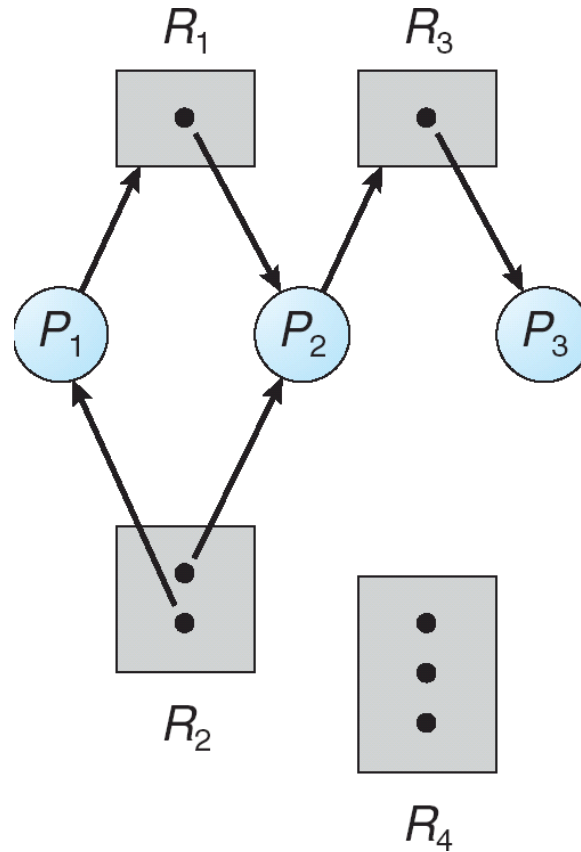
- P_i requests instance of R_j :



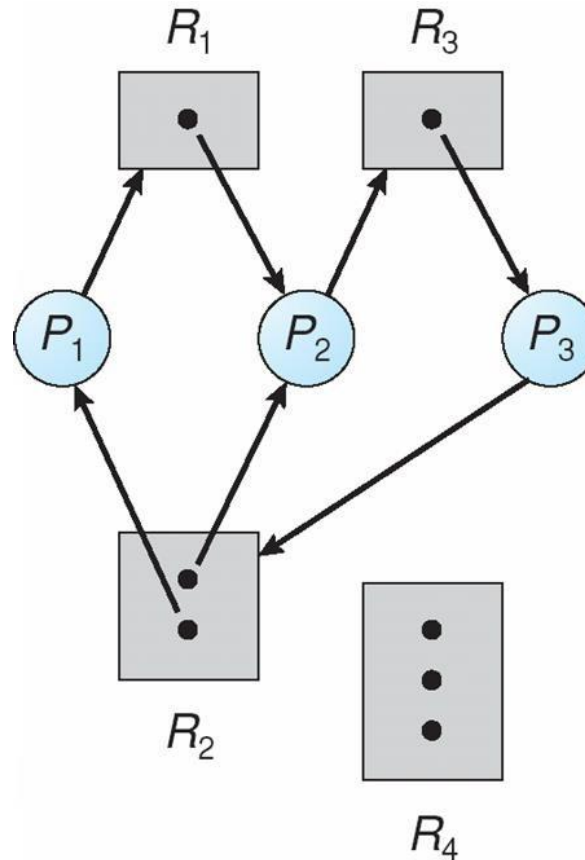
- P_i is holding an instance of R_j :



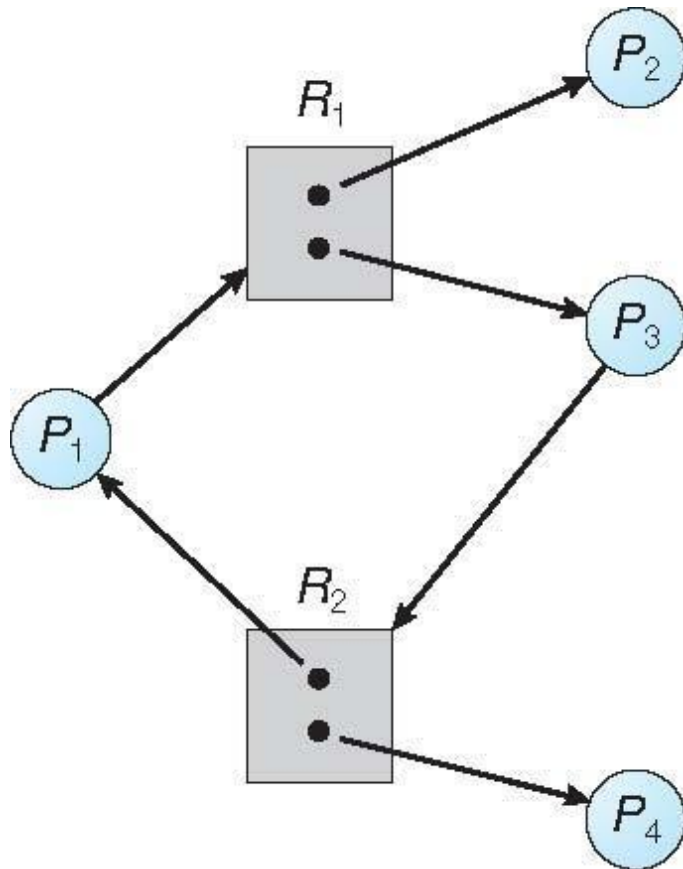
Example of a Resource Allocation Graph



Resource Allocation Graph With A Deadlock



Graph With A Cycle But No Deadlock



Process P_2 :

holds an instance of R_1
is NOT waiting for anything
So: ☒ P_2 can finish

Basic Facts

- If graph contains no cycles $\Rightarrow$ no deadlock.
- If graph contains a cycle $\Rightarrow$
 - if only one instance per resource type, then deadlock.
 - if several instances per resource type, possibility of deadlock.

Methods for Handling Deadlocks

- Ensure that the system will **never** enter a deadlock state.
- Allow the system to enter a deadlock state and then recover.
- **Ignore the problem** and pretend that deadlocks never occur in the system; used by some operating systems, including **UNIX**.

The Deadlock Ignorance strategy simply assumes:

- That deadlocks are so rare that it is not worth the cost of preventing or detecting them.
- If a deadlock does occur, the operating system may take drastic measures, such as rebooting to recover.
- This approach is called the **Ostrich Algorithm**, because it resembles the ostrich burying its head in the sand and pretending the problem doesn't exist.

Deadlock Prevention

Restrain the ways request can be made:

- **Mutual Exclusion** – not required for sharable resources; must hold for non-sharable resources.
- **Hold and Wait** – must guarantee that whenever a process requests a resource, it does not hold any other resources.
 - Require process to request and be allocated all its resources before it begins execution, or allow process to request resources only when the process has none.
 - Low resource utilization; starvation possible.

Deadlock Prevention

Deadlock prevention ensures that the system never enters a deadlock state.

A deadlock can occur only if all four Coffman conditions hold simultaneously:

1. Mutual Exclusion
2. Hold and Wait
3. No Preemption
4. Circular Wait

If we can prevent at least one of these conditions, deadlock becomes impossible.

Attacking Mutual Exclusion Condition

If resources are sharable, processes do not need to wait for exclusive access.

- Mutual exclusion means only one process can use a resource at a time.
- If resources are made sharable, deadlock possibility decreases.

Example

- Read-only files can be shared safely among multiple processes.
- Printer spooling allows several processes to generate output simultaneously.

Printer Spooling Concept

- Processes write output to disk.
- Printer daemon handles actual printing.
- Only the daemon accesses the physical printer.

Advantage

- Reduces direct competition for printer resource.

Limitation

- Some resources cannot be shared.
- Example: two processes writing to the printer simultaneously causes chaos

Attacking Hold-and-Wait Condition

Prevent processes from holding resources while waiting for additional resources.

Method 1: Request All Resources Initially

A process must request all required resources before execution starts.

Working

- If all resources are available → allocate all resources.
- Otherwise → process waits.

Advantages

- Eliminates hold-and-wait condition.
- Prevents deadlock.

Disadvantages

- Poor resource utilization.
- Processes may hold resources unnecessarily.
- Many processes do not know future resource needs.

Example

A process may reserve:

- printer
- plotter

long before actually using them.

Alternative Hold-and-Wait Solution

A process requesting a new resource must release all currently held resources before it can acquire the new resource.

Working

1. Process releases all resources.
2. Request all required resources again.
3. Continues execution only if all resources become available.

Advantage

- Eliminates circular resource dependency.

Limitation

- Frequent releasing and reacquiring causes overhead.
- Inefficient for long-running processes.

Attacking No-Preemption Condition

Allow resources to be preempted or virtualized. Normally, resources cannot be forcibly removed from a process.

Example

- A printer in the middle of printing cannot safely be taken away.

Virtualization Approach

Use virtual resources.

Example

Printer spooling:

- Processes store print jobs on disk.
- Printer daemon handles actual printing.

Advantage

- Reduces direct hardware contention.

Limitation

Some resources cannot be virtualized:

- Records in databases or tables inside the operating system must be locked to be used, and therein lies the potential for deadlock.

Attacking Circular Wait Condition

Prevent cyclic waiting among processes.

One Resource at a Time

A process may hold only one resource at a time.

Limitation

Impractical for many real systems.

Example

Copying a file from the **tape drive** to **the printer** would become impossible.

Resource Ordering Technique

Assign a numerical order to all resources.

Example Ordering

1. Imagesetter
2. Printer
3. Plotter
4. Tape Drive
5. Blu-ray Drive

Allowed

Printer → Tape Drive

Not Allowed

Plotter → Printer

Why It Works

- Cycles cannot form.
- Circular wait condition is eliminated.

Result

Deadlock becomes impossible.

Rule

Processes must request resources only in increasing order.

Why Resource Ordering Prevents Deadlock

Suppose:

- Process A holds resource i
- Process B holds resource j

If $i > j$:

- Process A cannot request j .

If $i < j$:

- Process B cannot request i .

Therefore:

- Circular dependency cannot occur.

No cycle $\rightarrow$ No deadlock.